Lab A: RTL kernel workflow and Mixing C++ and RTL Kernels

Content

- RTL kernel workflow
- Mixing C++ and RTL kernels
- Issues unsolved

Steps to generate the necessary RTL execution file

- Step 1: Read RTL files

```
#*/

set path_to_hdl "./src/IP"
set path_to_packaged "./packaged_kernel_${suffix}"
set path_to_tmp_project "./tmp_kernel_pack_${suffix}"

create_project -force kernel_pack $path_to_tmp_project
add_files -norecurse [glob $path_to_hdl/*.v $path_to_hdl/*.sv]
```

- Step 2: AXI interface setting

```
ipx::associate_bus_interfaces -busif m00_axi -clock ap_clk [ipx::current_core]
ipx::associate_bus_interfaces -busif m01_axi -clock ap_clk [ipx::current_core]
ipx::associate_bus_interfaces -busif s_axi_control -clock ap_clk [ipx::current_core]
```

```
module Vadd_A_B #(
  parameter integer C_S_AXI_CONTROL_ADDR_WIDTH = 12 ,
  parameter integer C_S_AXI_CONTROL_DATA_WIDTH = 32 ,
  parameter integer C_M00_AXI_ADDR_WIDTH       = 64 ,
  parameter integer C_M00_AXI_DATA_WIDTH       = 512,
  parameter integer C_M01_AXI_ADDR_WIDTH       = 64 ,
  parameter integer C_M01_AXI_DATA_WIDTH       = 512
)
```

Steps to generate the necessary RTL execution file (conti.)

- Step 3: generate the xo file

```
#package_xo -xo_path ${xoname} -kernel_name Vadd_A_B -ip_directory ./packaged_kernel_${suffix} -kernel_xml ./src/xml/kernel.xml -kernel_files ./src/c-
model/Vadd_A_B.cpp
package_xo -xo_path ${xoname} -kernel_name Vadd_A_B -ip_directory ./packaged_kernel_${suffix} -kernel_xml ./src/xml/user.xml -kernel_files ./src/c-mod
el/Vadd_A_B.cpp
~
~
~
```

- Step4: generate xclbin file

```
.PHONY: xclbin
xclbin: $(XCLBIN)/vadd.$(TARGET).xclbin

# Building kernel
$(XCLBIN)/vadd.$(TARGET).xclbin: $(BINARY_CONTAINER_vadd_OBJS)
    mkdir -p $(XCLBIN)
    $(VPP) $(CLFLAGS) $(LDCLFLAGS) -l -o $(XCLBIN)/vadd.$(TARGET).xclbin $(XO)/vadd.xo
    $(CP) $(XCLBIN)/vadd.$(TARGET).xclbin ./vadd.xclbin
```

Steps to generate the necessary files

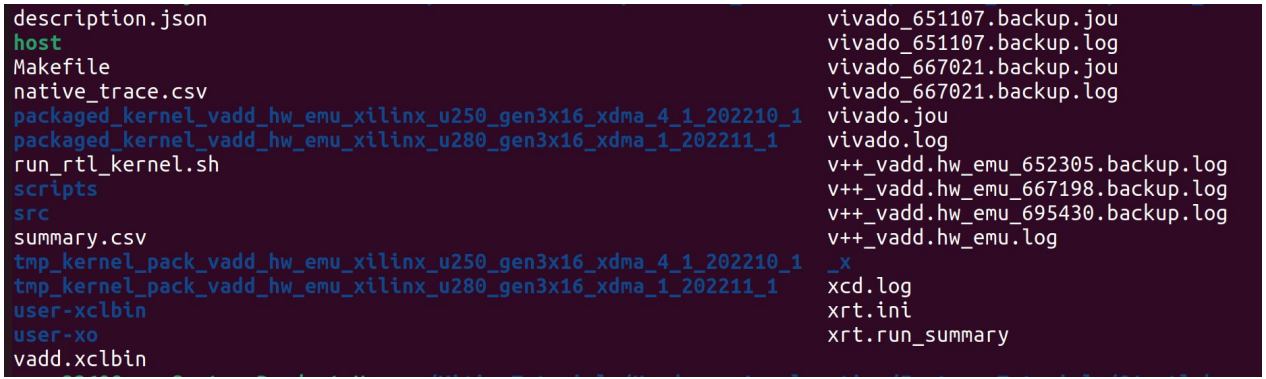
- Step 1: Enter the directory

  $ cd /Vitis-Tutorials/Hardware_Acceleration/Feature_Tutorials/01-rtl_kernel_workflow/reference-files

- Step 2:

  $ make all TARGET=xilinx_u280_gen3x16_xdma_1_202211_1

- Result:



Take a look at the time trace result

- open vitis analyzer

  $ vitis_analyze ./xrt.run_summary

- Notes for some tips:
  - If generate from the remote server, scp xrt.run_summary and summary.csv to generate the desired results

Host code

- src/host/user-host.cpp
- Step 1:

```
std::cout << "Allocate Buffer in Global Memory\n";
auto ip1_boA = xrt::bo(device, vector_size_bytes, 1);
auto ip1_boB = xrt::bo(device, vector_size_bytes, 1);

// Map the contents of the buffer object into host memory
auto bo0_map = ip1_boA.map<int*>();
auto bo1_map = ip1_boB.map<int*>();
```

- Step 2:

```
// Synchronize buffer content with device side
std::cout << "synchronize input buffer data to device global memory\n";
ip1_boA.sync(XCL_BO_SYNC_BO_TO_DEVICE);
ip1_boB.sync(XCL_BO_SYNC_BO_TO_DEVICE);

std::cout << "INFO: Setting IP Data" << std::endl;
std::cout << "Setting Register \"A\" (Input Address)" << std::endl;
ip1.write_register(A_OFFSET, buf_addr[0]);
ip1.write_register(A_OFFSET + 4, buf_addr[0] >> 32);

std::cout << "Setting Register \"B\" (Input Address)" << std::endl;
ip1.write_register(B_OFFSET, buf_addr[1]);
ip1.write_register(B_OFFSET + 4, buf_addr[1] >> 32);

uint32_t axi_ctrl = IP_START;
std::cout << "INFO: IP Start" << std::endl;
//axi_ctrl = IP_START;
ip1.write_register(USER_OFFSET, axi_ctrl);
```

Host code

- Step 3:

```
int i = 0;
while (krnl_done != IP_DONE) {
    axi_ctrl = ip1.read_register(USER_OFFSET);
    krnl_done = axi_ctrl & 0xffffffff2;
    krnl_idle = axi_ctrl & 0xffffffff4;
    i = i + 1;
    std::cout << "Read Loop iteration: " << i << " K
}

std::cout << "INFO: IP Done" << std::endl;
```

Mixing C++ and RTL kernels

- hw_emu

```
make all TARGET=hw_emu LAB=run2
export XCL_EMULATION_MODE=hw_emu
./host_2 krnl_vadd.hw_emu.xclbin
```

- sw_emu

```
make all TARGET=sw_emu LAB=run1
export XCL_EMULATION_MODE=sw_emu
./host_1 krnl_vadd.hw_emu.xclbin
```

Modify Makefile & xrt.ini

- Makefile

```
# Building kernel
$(XO): ./src/kernel_cpp/krnl_vadd.cpp
ifeq ($(TARGET),sw_emu)
    v++ $(CLFLAGS) -c -k krnl_vadd -g -o'$@' '$<'
else
    v++ -c -k krnl_vadd     --platform xilinx_u280_gen3x16_xdma_1_202211_1      --config hls_config.cfg      -o krnl_vadd.hw_emu.xo      src/kernel_cp
p/krnl_vadd.cpp
    #v++ --platform $(DEVICE) --config hls_config.cfg
    # --mode hls
```

- xrt.ini

```
[Debug]
native_xrt_trace=true
device_trace=fine
~
~
```

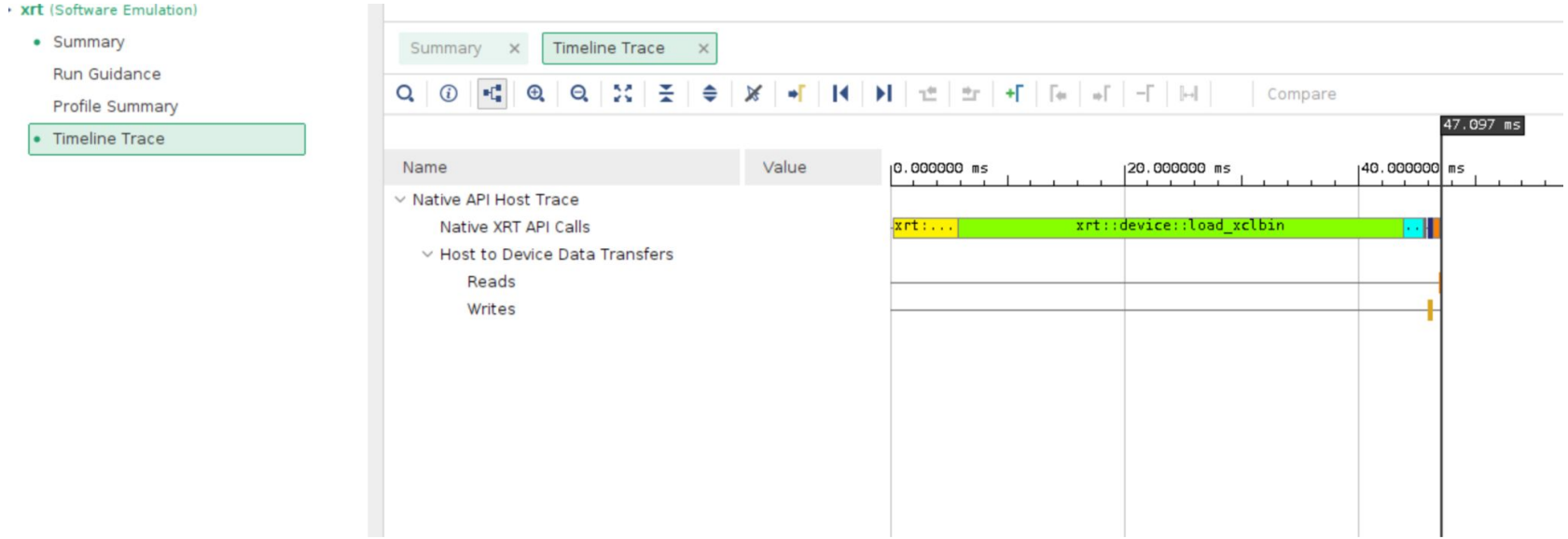
Software Emulation

```
.sw_emu.xclbin
argc = 2
argv[0] = ./host_1
argv[1] = krnl_vadd.sw_emu.xclbin
Open the device0
Load the xclbin krnl_vadd.sw_emu.xclbin
Kernel Name: krnl_vadd_1, CU Number: 0, Thread creation status: success
Allocate Buffer in Global Memory
synchronize input buffer data to device global memory
Execution of the kernel
Kernel Name: krnl_vadd_1, CU Number: 0, State: Start
Kernel Name: krnl_vadd_1, CU Number: 0, State: Running
Kernel Name: krnl_vadd_1, CU Number: 0, State: Idle
Get the output data from the device
TEST WITH ONE KERNEL PASSED
device process sw_emu_device done
Kernel Name: krnl_vadd_1, CU Number: 0, Status: Shutdown
```

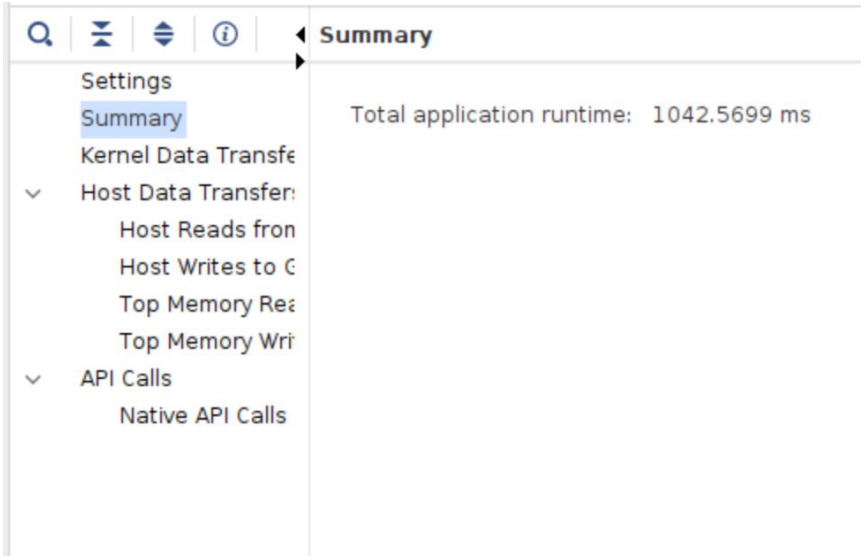
Hardware Emulation

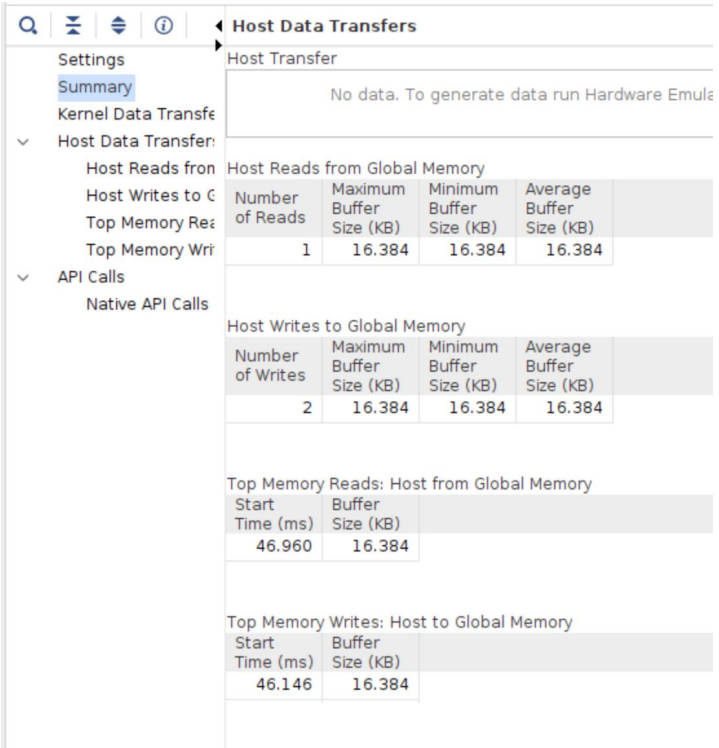
```
.hw_emu.xclbin
argc = 2
argv[0] = ./host_2
argv[1] = krnl_vadd.hw_emu.xclbin
Open the device0
Load the xclbin krnl_vadd.hw_emu.xclbin
INFO: [HW-EMU 05] Path of the simulation directory : /home/ramo3369/Vitis-Tutorials/Hardware_Acceleration/Feature_Tutorial
ference-files/.run/766502/hw_em/device0/binary_0/behav_waveform/xsim

 server socket name is  /tmp/ramo3369/device0_0_766502
INFO: [HW-EMU 01] Hardware emulation runs simulation underneath. Using a large data set will result in long simulation tim
 small dataset is used for faster execution. The flow uses approximate models for Global memories and interconnect and hen
rated is approximate.
configuring dataflow mode with ert polling
scheduler config ert(1), dataflow(1), slots(16), cudma(0), cuisr(0), cdma(0), cus(2)
Allocate Buffer in Global Memory
synchronize input buffer data to device global memory
Execution of the vadd kernel
Execution of the RTL  kernel
Get the output data from the device
TEST PASSED
INFO: [HW-EMU 06-0] Waiting for the simulator process to exit
INFO: [HW-EMU 06-1] All the simulator processes exited successfully
INFO: [HW-EMU 07-0] Please refer the path "/home/ramo3369/Vitis-Tutorials/Hardware_Acceleration/Feature_Tutorials/02-mixin
```

Vitis Analyzer (sw_emu)



Profiling Result





Issues Unsolved

- Generate the pure rtl execution file without external xo file (rtl_kernel_wizard_0.xo) and run it along with host kernel

Solved

```
all: $(XCLBIN) host

$(XO_FILE): scripts/gen_xo.tcl system/sram.sv system/sram_kernel.sv
    mkdir -p $(BUILD)
    $(VIVADO) -mode batch -source scripts/gen_xo.tcl \
      -tclargs $(XO_FILE) sram_kernel $(TARGET) $(DEVICE)

$(XCLBIN): $(XO_FILE)
    $(VPP) -l -t $(TARGET) \
      --platform $(DEVICE) \
      $(XO_FILE) \
      -o $@ \
      --save-temps

host: host.cpp
    $(CXX) -std=c++17 -I$(XILINX_XRT)/include -L$(XILINX_XRT)/lib -lxrt_coreutil \
           host.cpp -o $@

run_emu:
    export XCL_EMULATION_MODE=$(TARGET); \
    ./host $(XCLBIN)

clean:
    rm -rf $(BUILD) host .Xil *.log *.jou *run_summary

~
~
~
```